

Create an Azure Cosmos DB Database and Container Using the .NET SDK

Summary:-

In this lab, you will create an Azure Cosmos DB database and container programmatically using the Azure Cosmos DB .NET SDK. You will create a .NET Console Application, install the required SDK package, connect to your Azure Cosmos DB account, and create a database and container using C# code instead of the Azure portal.

Let's Start:-

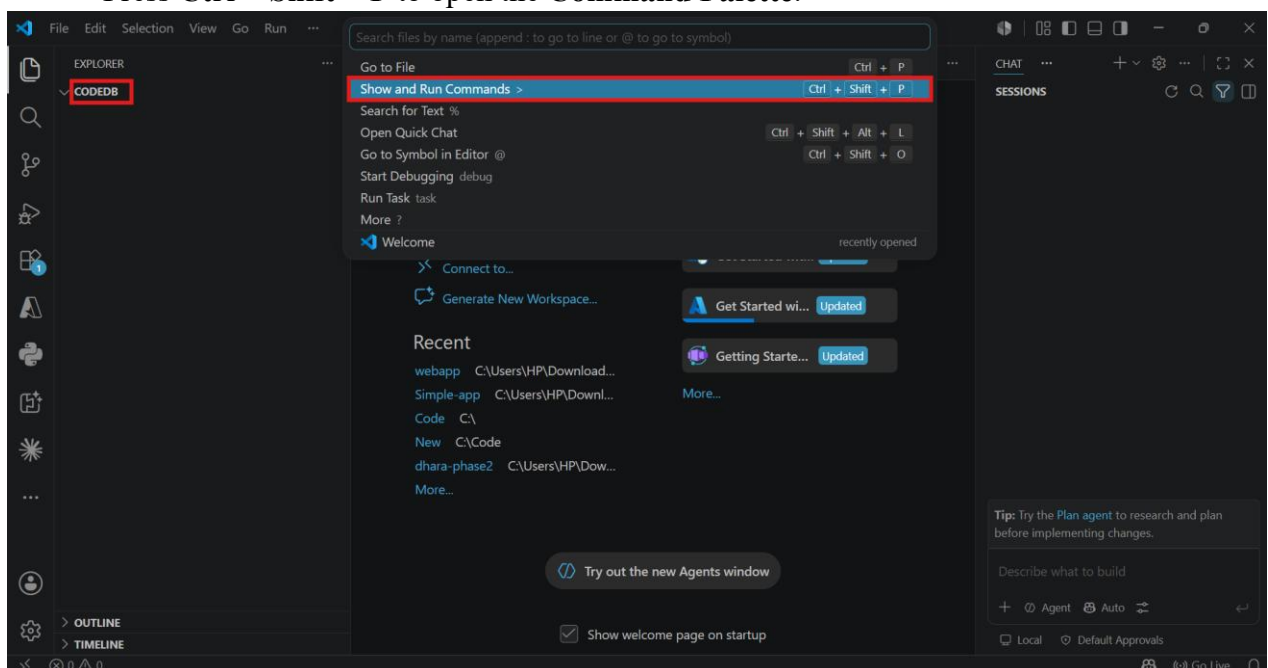
After completing this lab, you will be able to:

- Create a .NET Console Application.
- Install the Azure Cosmos DB .NET SDK.
- Connect a .NET application to Azure Cosmos DB.
- Create a database programmatically.
- Create a container with a partition key using the .NET SDK.

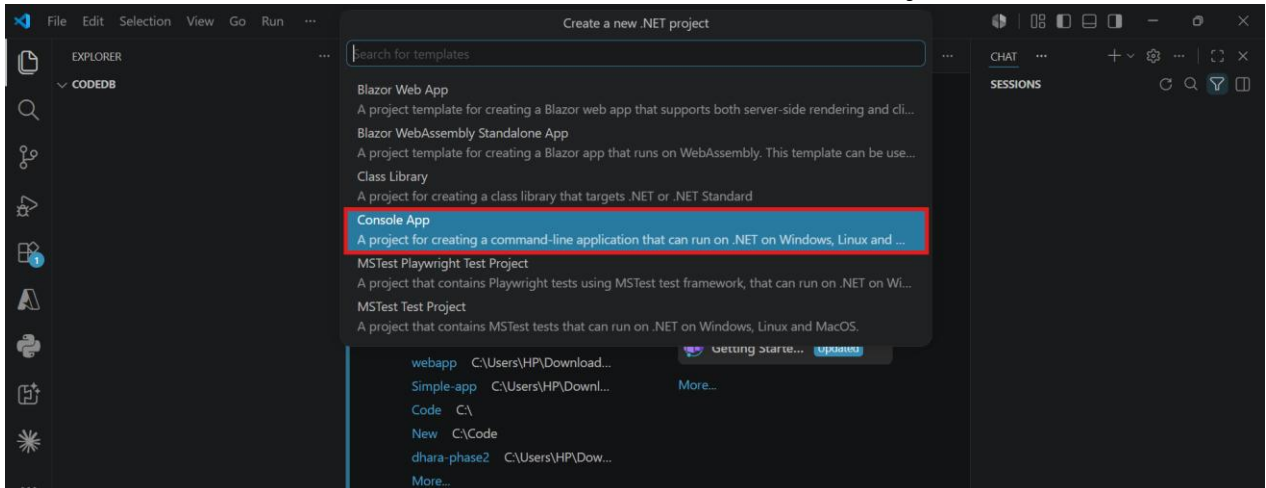
Lab Steps:-

Step 1 – Create a .NET Console Application

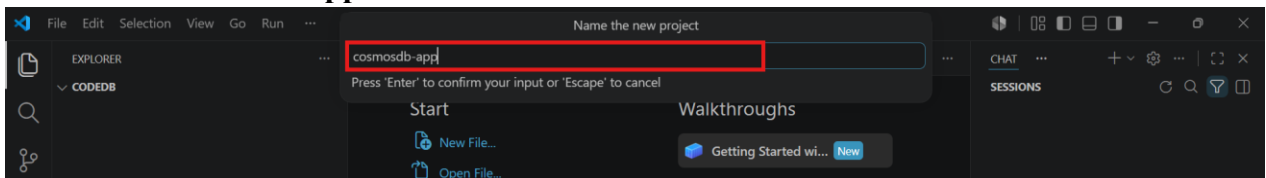
- Open Visual Studio Code.
- Create a new project folder.
- Press **Ctrl + Shift + P** to open the Command Palette.



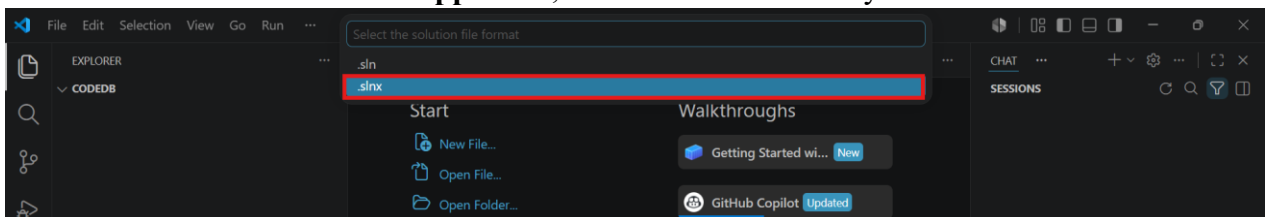
- Select **Show and Run Commands** >. Choose **.NET: New Project**.



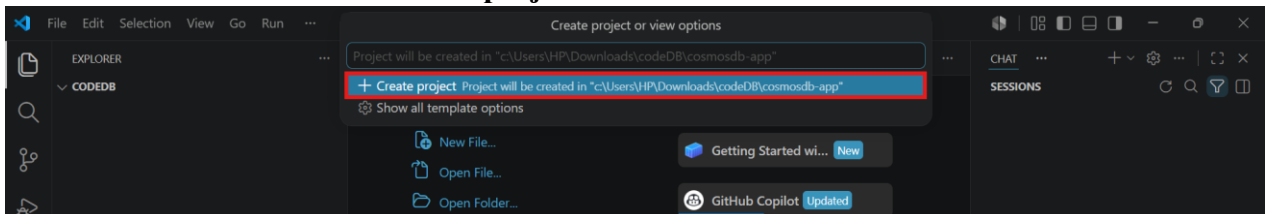
- Select **Console App**.



- Rename it as **cosmosdb-app**. Then, Select Default Directory.



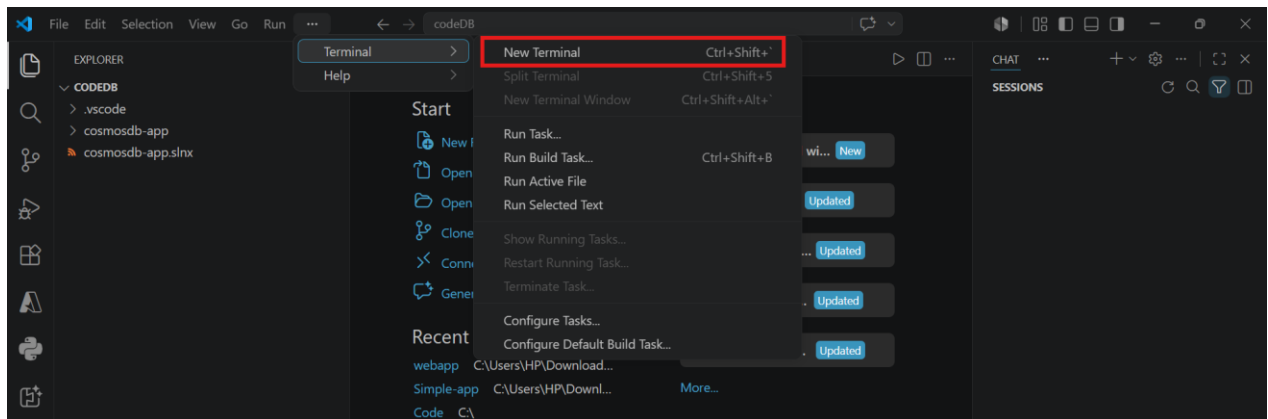
- Select **.slnx**. Click on **Create project**.



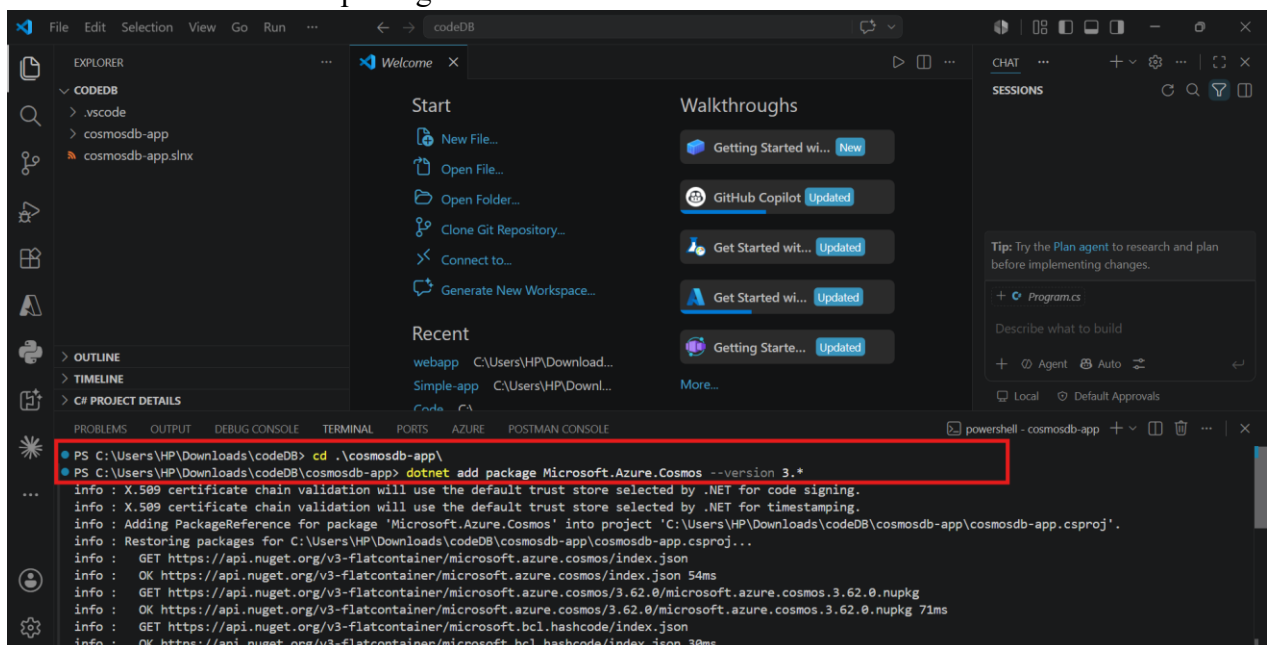
- This step creates a .NET Console Application for interacting with Azure Cosmos DB.

Step 2 – Install the Azure Cosmos DB SDK

- Open the integrated terminal.



- Run command:
`cd .\cosmosdb-app\`
- Install the Azure Cosmos DB SDK by running:
`dotnet add package Microsoft.Azure.Cosmos`
`dotnet add package Newtonsoft.Json`



- Wait for the package installation to complete successfully.
- This step installs the Azure Cosmos DB .NET SDK required to interact with Cosmos DB.

Step 3 – Obtain the Cosmos DB Endpoint and Key

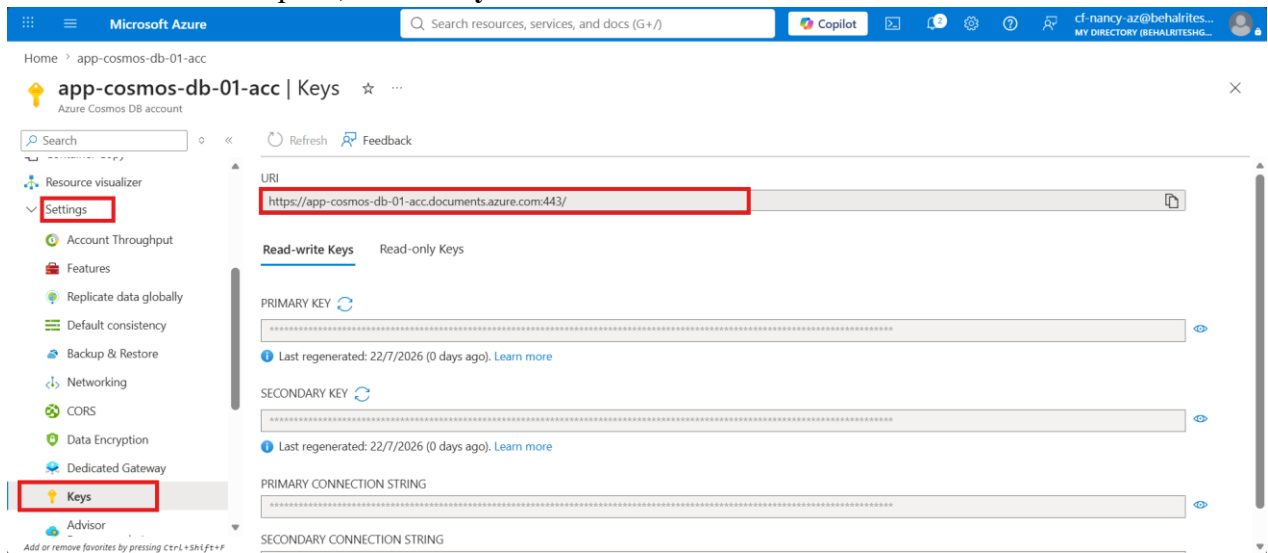
- Sign in to the **Azure portal**.
- **Create** Azure Cosmos DB account (based on previous labs) by selecting **Azure Cosmos DB for NoSQL** from the recommended APIs.
- Configure the account with the required settings:
Workload type: Learning

Region: EAST US/ EAST US 2

Capacity Mode: Provisioned Throughput

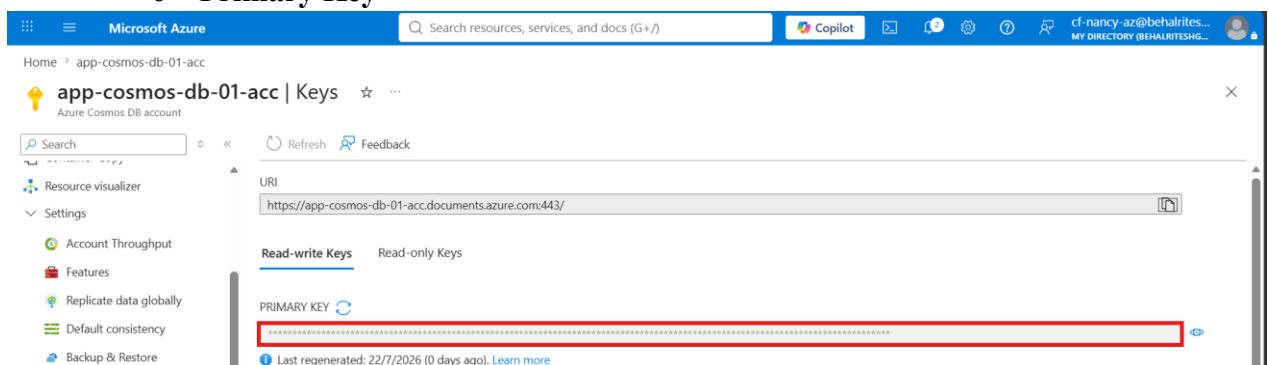
Enable **Apply Free Tier Discount**. Click **Next**.

- Ensure Geo-Redundancy and Multi-region Writes is **Disabled**. Click **Next**.
- Select **Enable access from all networks**. Click on **Review + Create** to create.
- From the left pane, select **Keys**.

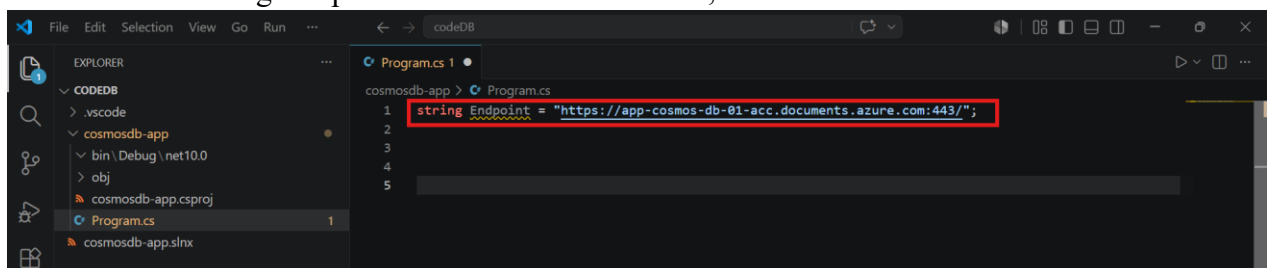


- Copy the following values:

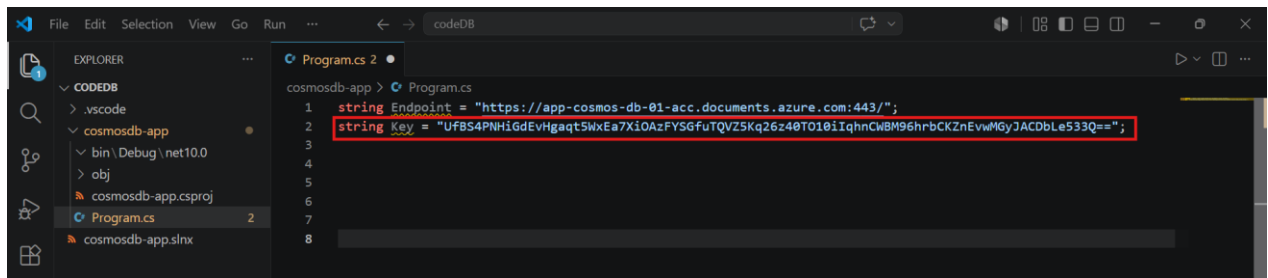
- **URI**
- **Primary Key**



- Open **Program.cs**.
- Create variables to store the endpoint and key.
string Endpoint = "<CosmosDB-URI>";



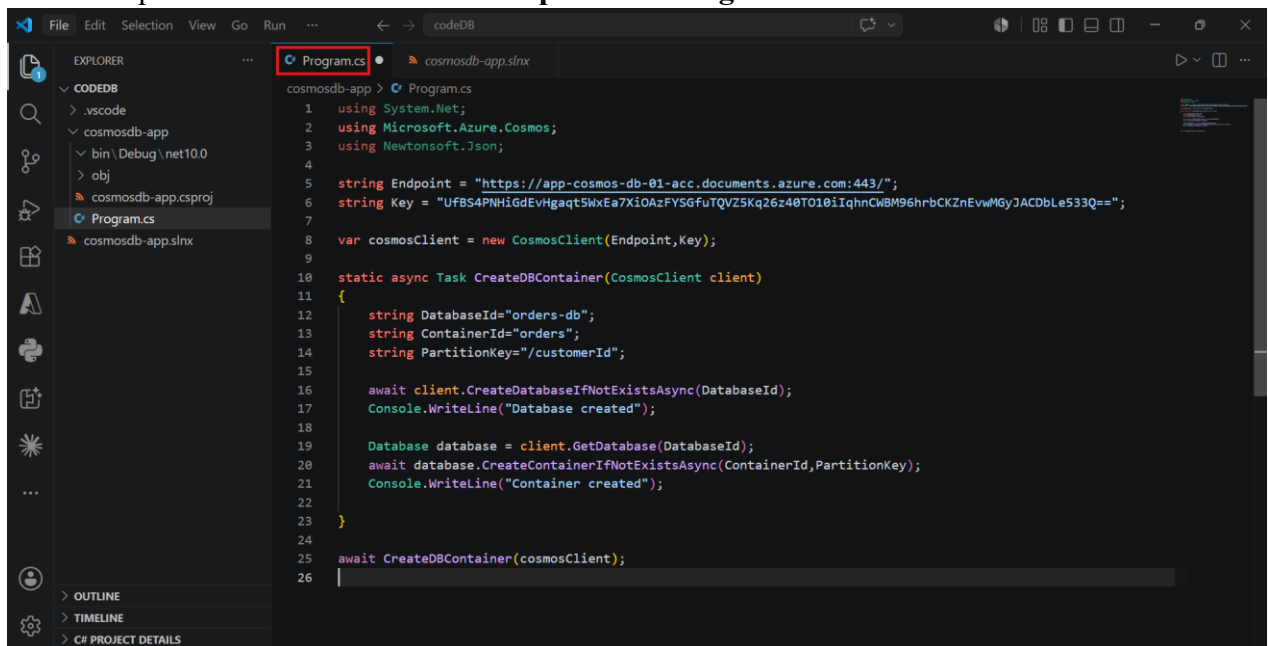
string key = "<Primary-Key>";



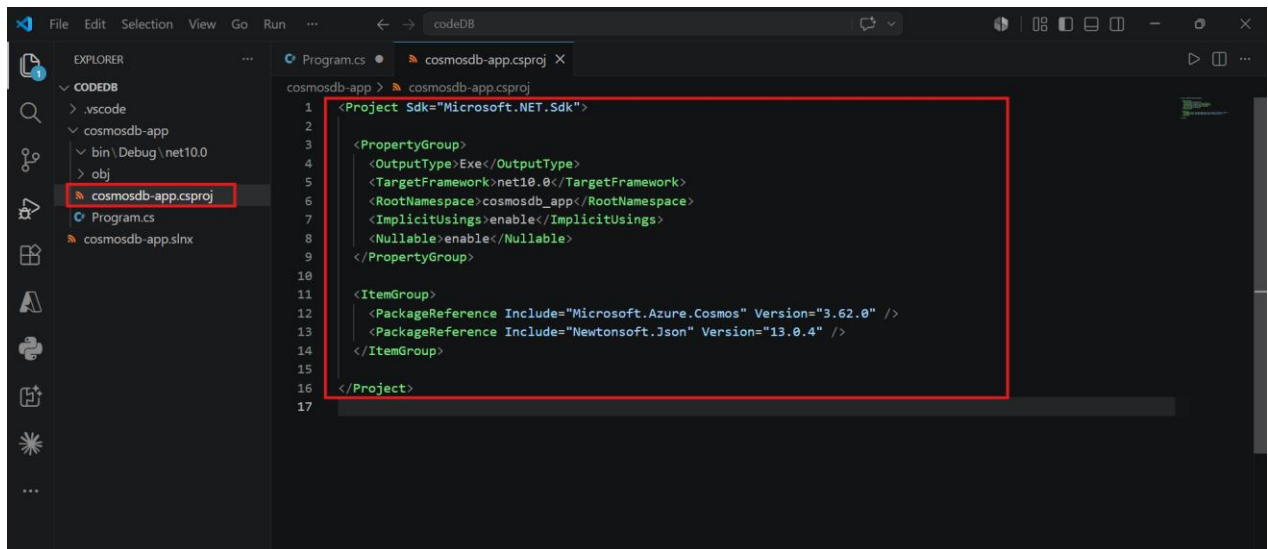
- This step prepares the application to connect to Azure Cosmos DB.

Step 4 – Add the .NET Code

- Open **Program.cs**.
- Replace the default code with the **provided Program.cs** code.



- Replace the default code with the **provided cosmosdb-app.csproj** code.



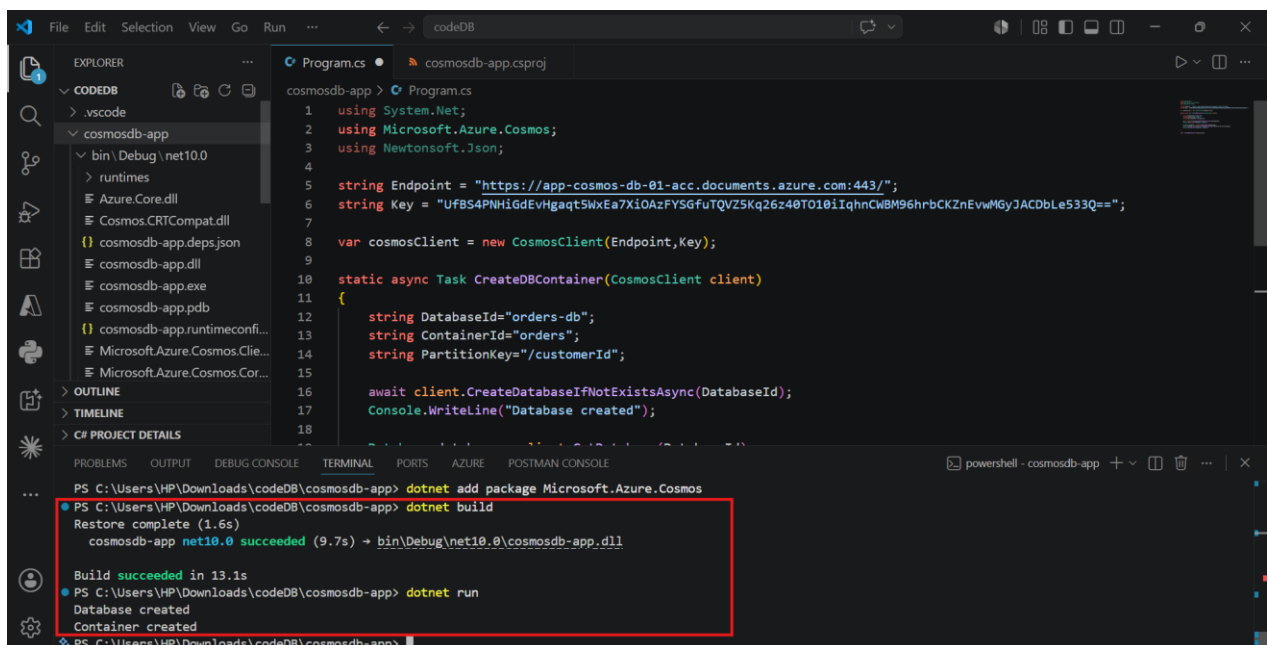
- This step prepares the application to create the Azure Cosmos DB database and container programmatically.

Step 5 – Run the Application

- Open the **Terminal** in Visual Studio Code.
- Ensure you are in the project directory.
- **Run** the following command:

```
dotnet build
```

```
dotnet run
```



- Wait for the application to execute successfully.
- Verify that the following **messages are displayed** in the terminal:

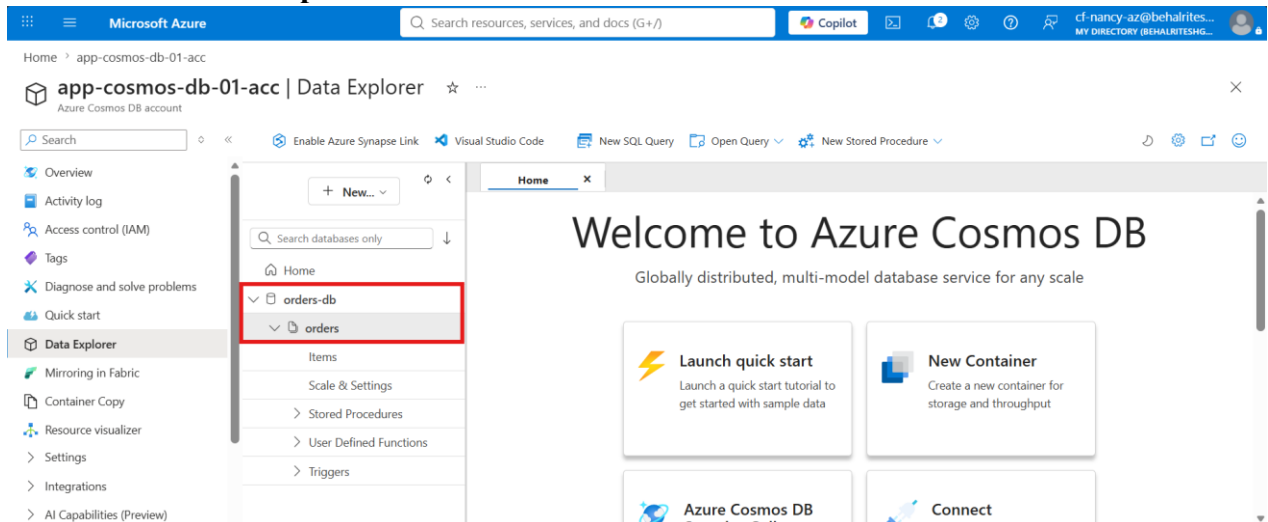
```
Database created
```

Container created

- This step executes the .NET application to create the database and container.

Step 6 – Verify the Resources

- Sign in to the Azure Portal.
- Open your **Azure Cosmos DB** account.
- Select **Data Explorer**.



- Verify that the **orders-db** database has been created.
- Expand the database and verify that the **orders** container is available.
- This step verifies that the database and container were successfully created using the Azure Cosmos DB .NET SDK.

Verification

Verify the following:

- The .NET application executes successfully without errors.
- The database is created in the Azure Cosmos DB account.
- The container is created under the database.
- The configured partition key is applied to the container.
- The created database and container are visible in the Azure Portal under **Data Explorer**.